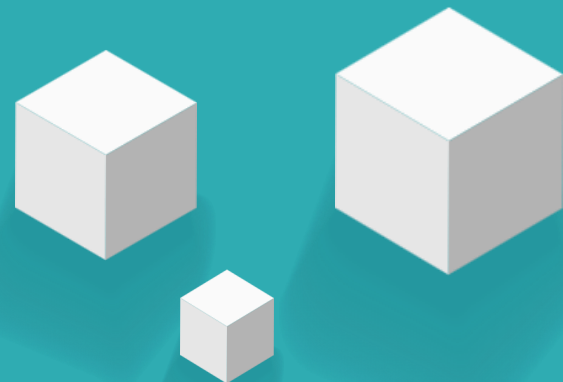
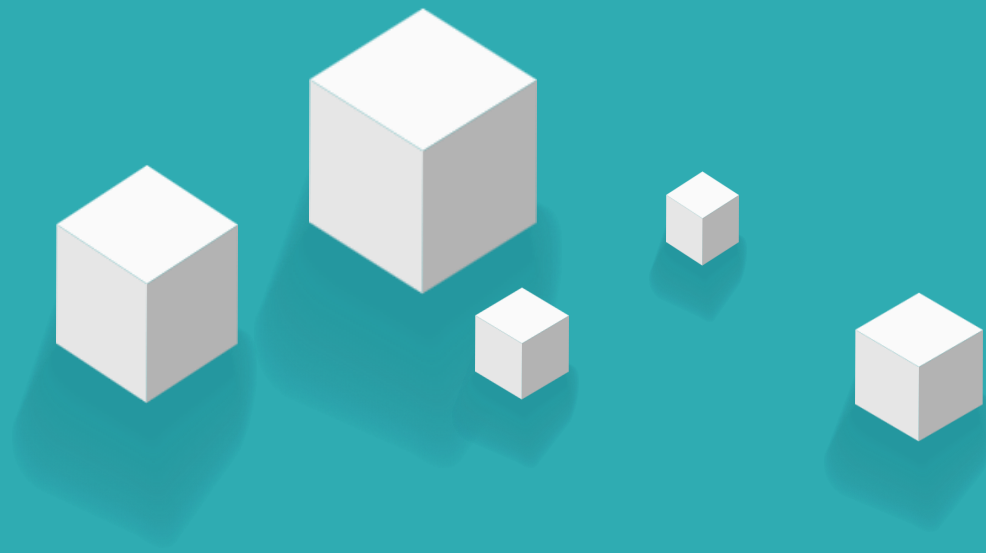


vue.js
Aug 12



第八章 组件基础 (下)





8.5

动态组件

8.6

插槽

8.7

自定义指令

8.8

引用静态资源

8.9

阶段案例



8.5

动态组件

» 8.5 动态组件

8.5.1 定义动态组件

- 利用动态组件可以动态切换页面中显示的组件。使用<component>标签可以定义动态组件，语法格式如下。

```
<component :is="要渲染的组件"></component>
```

■ 说明

- <component>标签必须配合is属性一起使用，is属性的属性值表示要渲染的组件。
- is属性的属性值：
 - 字符串
 - 组件：要实现组件的切换，需要调用shallowRef()函数定义响应式数据，将组件保存为响应式数据。



只处理对象最外层属性的响应，它比ref()函数更适合将组件保存为响应式数据。

如果有一个对象数据，后续功能不会修改该对象中的属性，而是生新的对象来替换==>shallowRef。

8.5 动态组件

※动态组件的使用方法示例※

①创建项目

```
npm create vite@4.1.0 component_foundation-- --template vue
```

②启动项目

```
cd component_foundation  
npm install  
npm run dev
```

③使用VS Code编辑器打开D:\...\component_foundation目录。

④将src\style.css文件中的样式代码全部删除。

⑤创建src\components\MyLeft.vue文件。

```
<template> MyLeft组件 </template>
```

⑥创建src\components\MyRight.vue文件。

```
<template> MyRight组件 </template>
```

▼ COMPONENT_FOUNDATION

- > .vscode
- > node_modules
- > public
- > src
- 📄 .gitignore
- <> index.html
- { } package.json
- 📖 README.md
- JS vite.config.js
- 🔒 yarn.lock

③

»» 8.5 动态组件

动态组件的使用方法示例

⑦创建src\components\DynamicComponent.vue文件，在该文件中导入并使用MyLeft和MyRight组件，实现单击按钮时动态切换组件的效果。

```
1  <template>
2    <button @click="showComponent = MyLeft">展示MyLeft组件</button>
3    <button @click="showComponent = MyRight">展示MyRight组件</button>
4    <div><component :is="showComponent"></component></div>
5  </template>
6  <script setup >
7    import MyLeft from './MyLeft.vue'
8    import MyRight from './MyRight.vue'
9    import { shallowRef } from 'vue'
10   const showComponent = shallowRef(MyLeft)
11  </script>
```



渲染动态组件

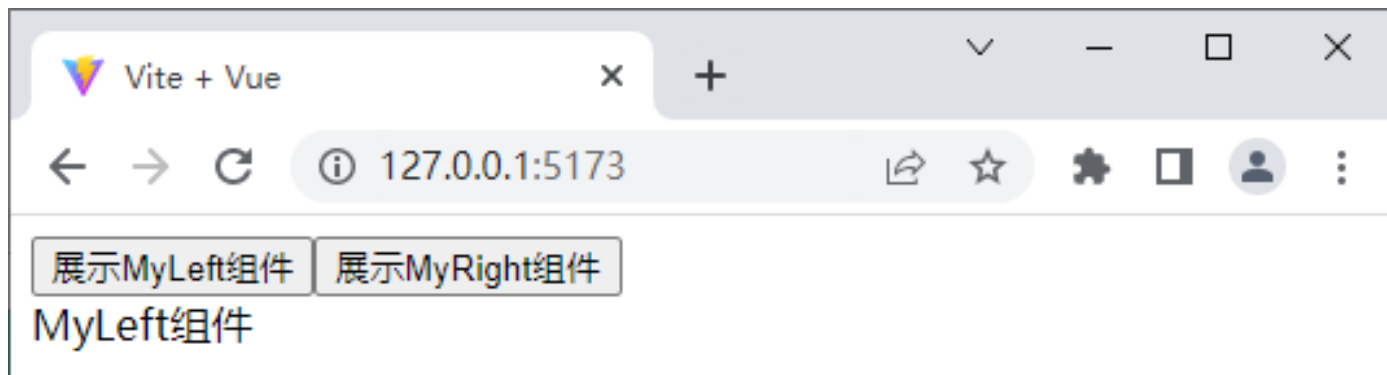
⑧修改src\main.js文件，切换页面中显示的组件。

```
import App from './components/DynamicComponent.vue'
```

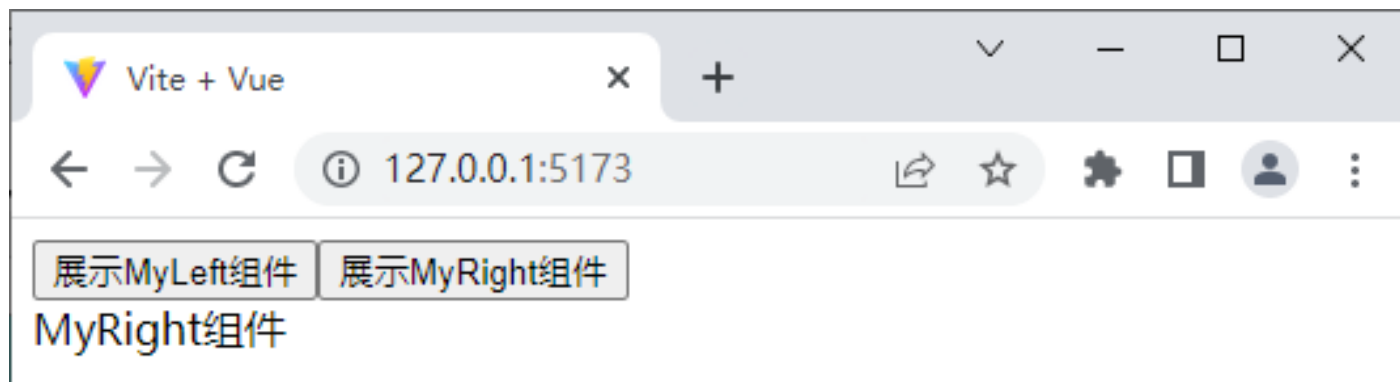
» 8.5 动态组件

动态组件的使用方法示例

在浏览器中访问<http://127.0.0.1:5173/>，动态组件渲染后的页面效果如下图所示。



单击“展示MyRight组件”按钮后的页面效果。



»» 8.5 动态组件

8.5.2 利用KeepAlive组件实现组件缓存

- 使用动态组件实现组件切换

- 隐藏的组件会被销毁。
- 展示出来的组件会被重新创建。
- 重新创建时无法保持销毁前的状态。

- 通过组件缓存来实现

- 多个组件之间进行动态切换时保持这些组件的状态。
- 避免重复渲染导致的性能问题。

» 8.5 动态组件

8.5.2 利用KeepAlive组件实现组件缓存

- 通过KeepAlive组件来实现组件缓存。
 - 组件缓存可以使组件创建一次后，不会被销毁。
- KeepAlive组件通过<KeepAlive>标签来定义，使用<KeepAlive>标签包裹需要被缓存的组件。
- <KeepAlive>标签的语法格式如下。

```
<KeepAlive>  
  被缓存的组件  
</KeepAlive>
```

»» 8.5 动态组件

※KeepAlive组件的使用示例※-1

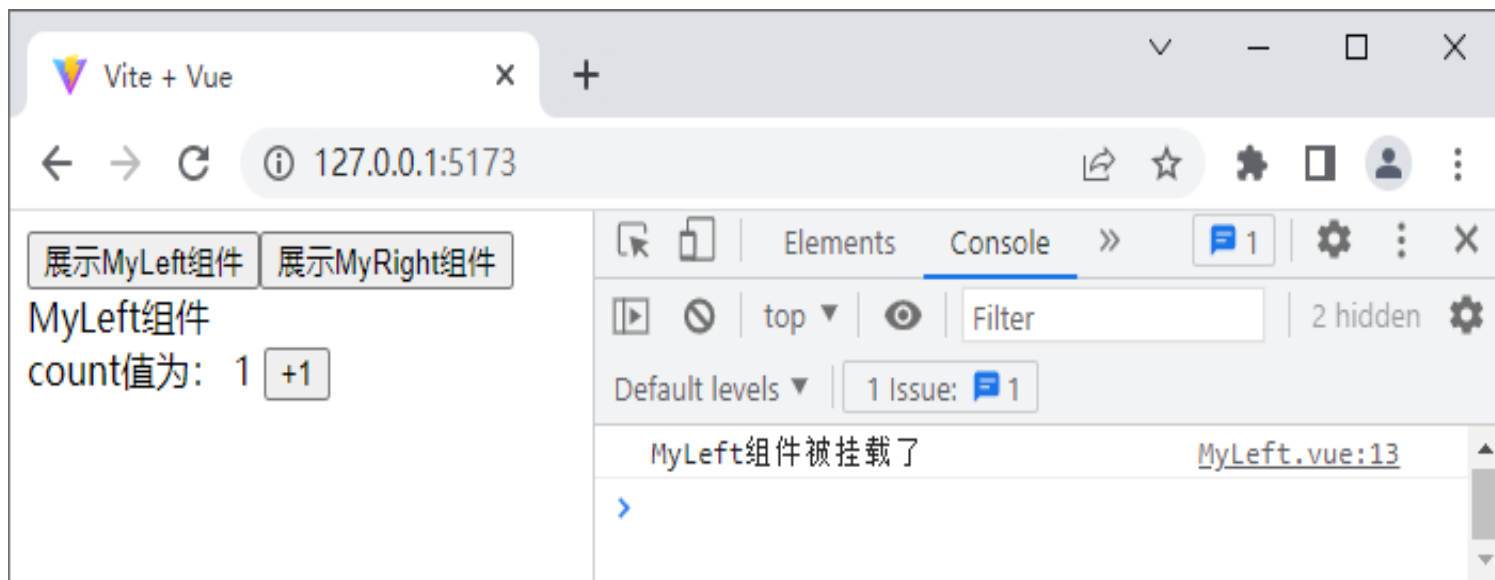
①编写src\components\MyLeft.vue文件（不使用KeepAlive组件），观察动态组件被销毁的过程。

```
1  <template>
2    MyLeft组件
3    <div>
4      count值为:  {{ count }}
5      <button @click="count++">+1</button>
6    </div>
7  </template>
8  <script setup>
9    import { ref, onMounted, onUnmounted } from 'vue'
10   const count = ref(0)
11   onMounted(() => {
12     console.log('MyLeft组件被挂载了')
13   })
14   onUnmounted(() => {
15     console.log('MyLeft组件被销毁了')
16   })
17 </script>
```

8.5 动态组件

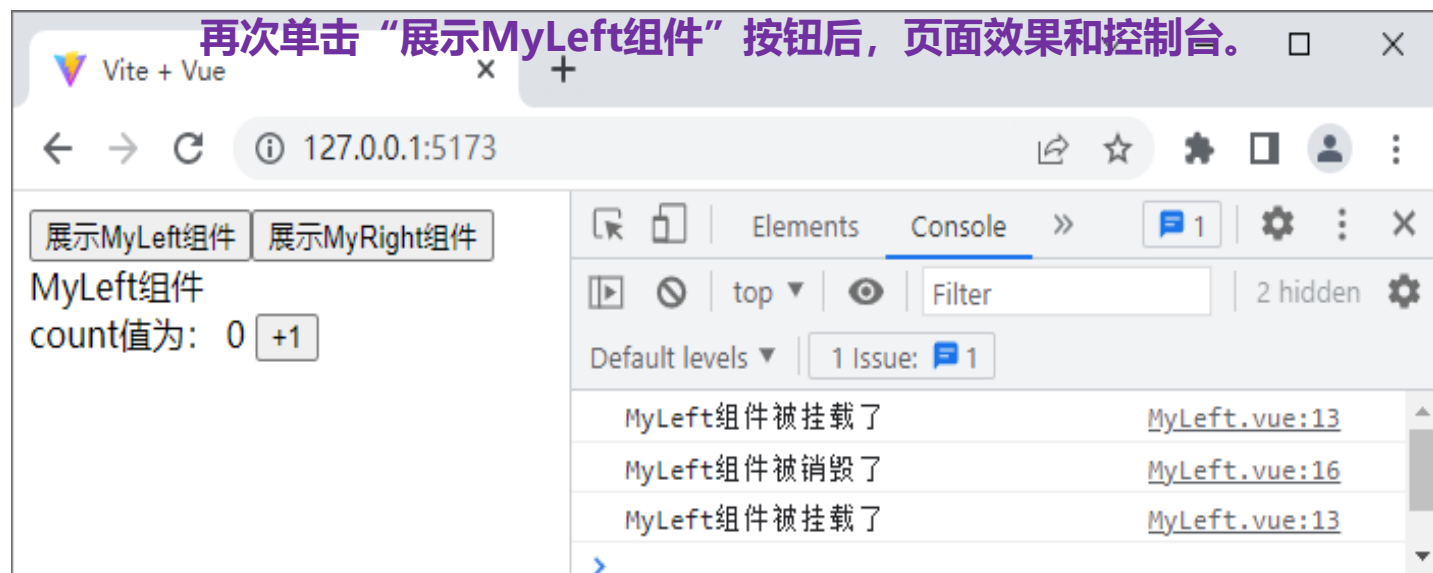
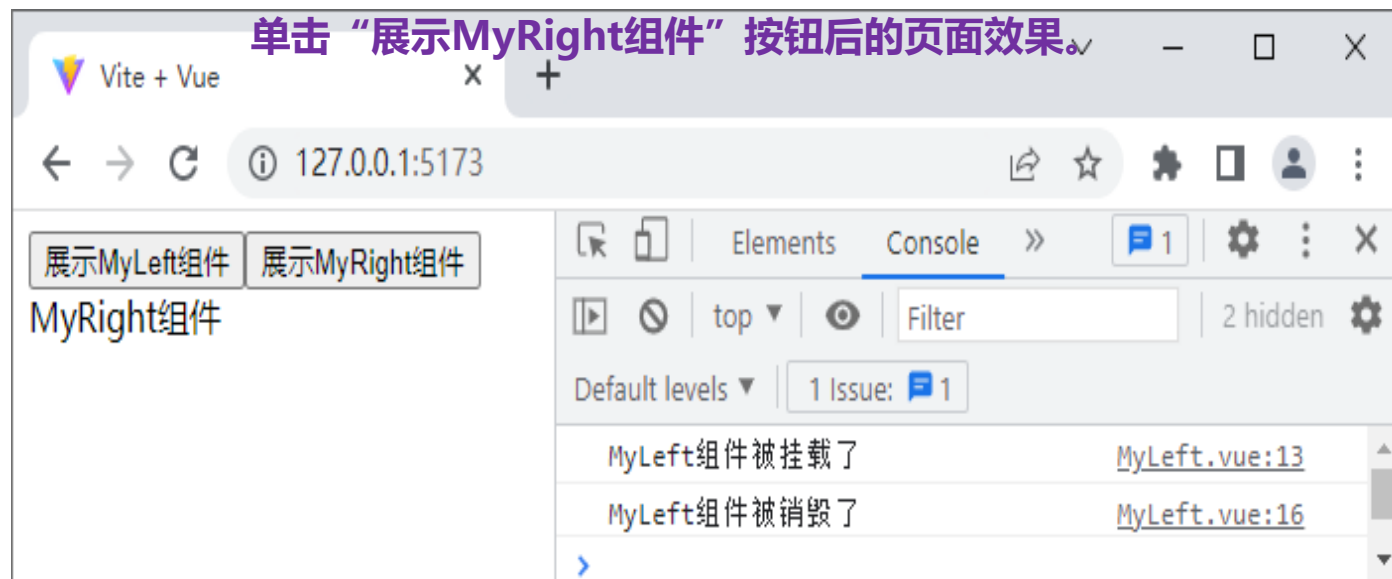
※KeepAlive组件的使用示例※-1

在浏览器中访问<http://127.0.0.1:5173/>，单击“+1”按钮后的页面效果和控制台如下图所示。



>>> 8.5 动态组件

※KeepAlive组件的使用示例※-1



8.5 动态组件

※KeepAlive组件的使用示例※-2

② 修改src\components\DynamicComponent.vue文件，添加<KeepAlive>标签将实现组件缓存。

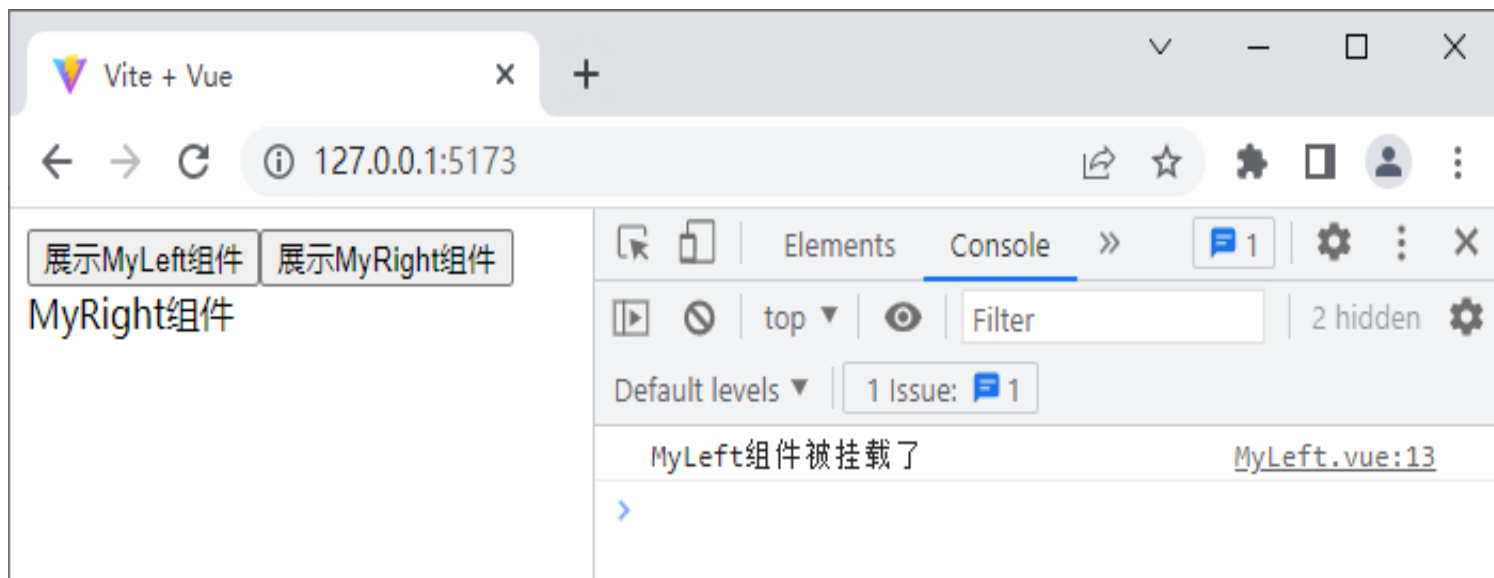
```
<div>
  <KeepAlive>
    <component :is="showComponent"> </component>
  </KeepAlive>
</div>
```

```
1  <template>
2    <button @click="showComponent = MyLeft">展示MyLeft组件</button>
3    <button @click="showComponent = MyRight">展示MyRight组件</button>
4    <div><component :is="showComponent"></component></div>
5  </template>
6  <script setup >
7    import MyLeft from './MyLeft.vue'
8    import MyRight from './MyRight.vue'
9    import { shallowRef } from 'vue'
10   const showComponent = shallowRef(MyLeft)
11  </script>
```

8.5 动态组件

※KeepAlive组件的使用示例※-2

浏览器中访问<http://127.0.0.1:5173/>



- ◆ 单击 “+1” 按钮后的页面效果与示例1中单击 “+1” 按钮后的页面效果相同。
- ◆ 单击 “展示MyRight组件” 按钮后，控制台中没有输出 “MyLeft组件被销毁了” 的信息，说明此时MyLeft组件已经被缓存了。
- ◆ 再次单击 “展示MyLeft组件” 按钮，页面效果与示例1中单击 “+1” 按钮后的页面效果相同，说明通过添加<KeepAlive> 标签可以实现组件缓存。



※KeepAlive组件的使用示例※-2

Vite App

http://localhost:5173

120%

介绍 | Vue.js 框架 工具 | Vue.js 框架 UNPKG - vue vue CDN Vue.js 官网

展示MyLeft组件 展示MyRight组件

MyLeft组件
count值为: 0 +1

查看器 控制台 调试器 网络

过滤输出

错误 警告 信息 日志 调试 CSS XHR 请求

[vite] connecting... client:742:8

[vite] connected. client:865:14

MyLeft组件被挂载了 MyLeft.vue:12:10

» 8.5 动态组件

8.5.3 组件缓存相关的生命周期函数

- Vue提供了组件缓存相关的生命周期函数，可以监听组件被激活和组件被缓存的事件。
 - `onActivated()`：组件被激活时触发。
 - `onDeactivated()`：组件被缓存时触发。
- 这两个生命周期函数的语法格式如下。

```
// onActivated()生命周期函数
```

```
onActivated() => { }
```

```
// onDeactivated()生命周期函数
```

```
onDeactivated() => { }
```


»» 8.5 动态组件

组件缓存相关的生命周期函数的使用示例

①修改 MyLeft.vue文件，在<script>标签中定义onActivated()和onDeactivated()生命周期函数。

```
1  <script setup>
2  import { ref, onMounted, onUnmounted, onActivated, onDeactivated } from 'vue'
3  onActivated(() => {
4    | console.log('MyLeft组件被激活了')
5  })
6  onDeactivated(() => {
7    | console.log('MyLeft组件被缓存了')
8  })
9  </script>
```

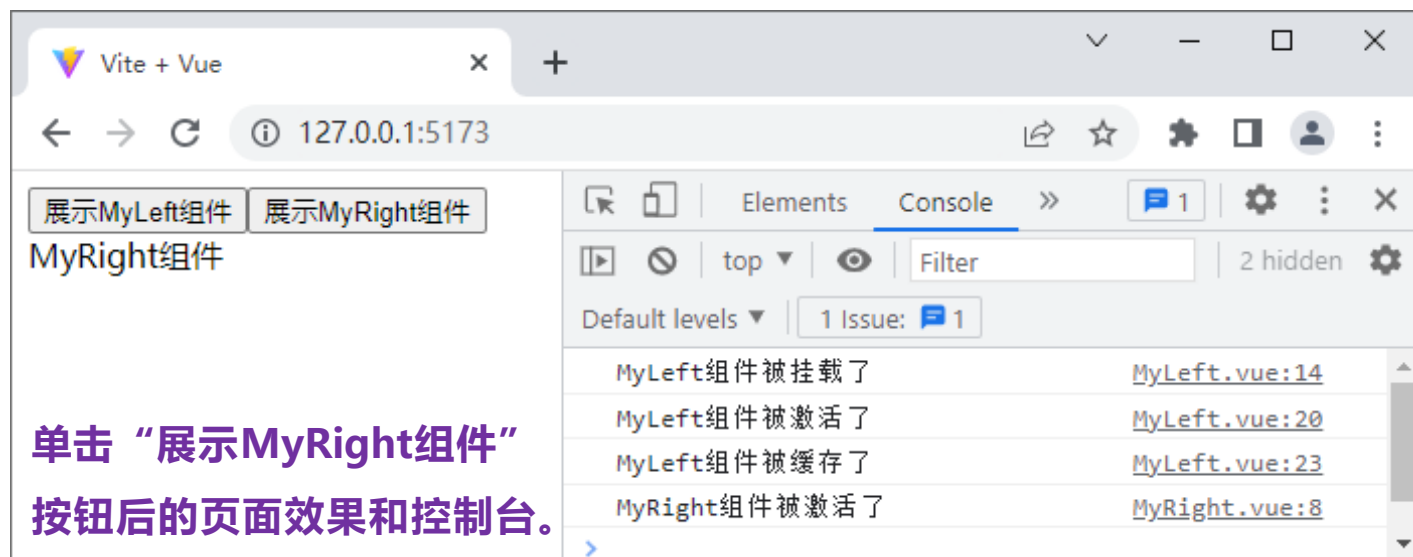
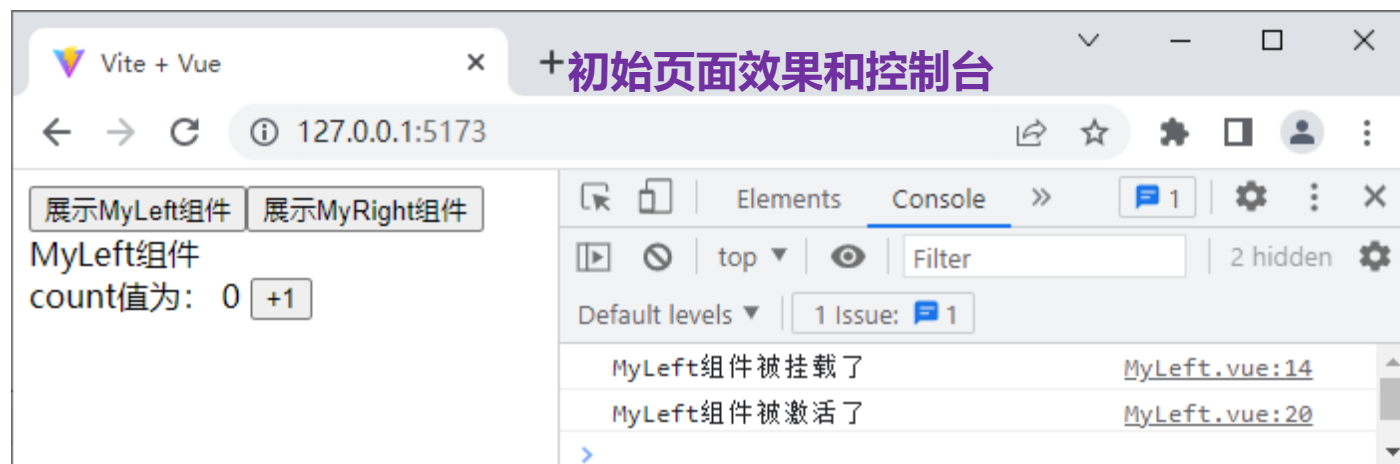
②修改MyRight.vue文件，
在<script>标签中定义
onActivated()和
onDeactivated()生命周期
函数。

```
1  <script setup>
2  import { onActivated, onDeactivated } from 'vue'
3  onActivated(() => {
4    | console.log('MyRight组件被激活了')
5  })
6  onDeactivated(() => {
7    | console.log('MyRight组件被缓存了')
8  })
9  </script>
```

8.5 动态组件

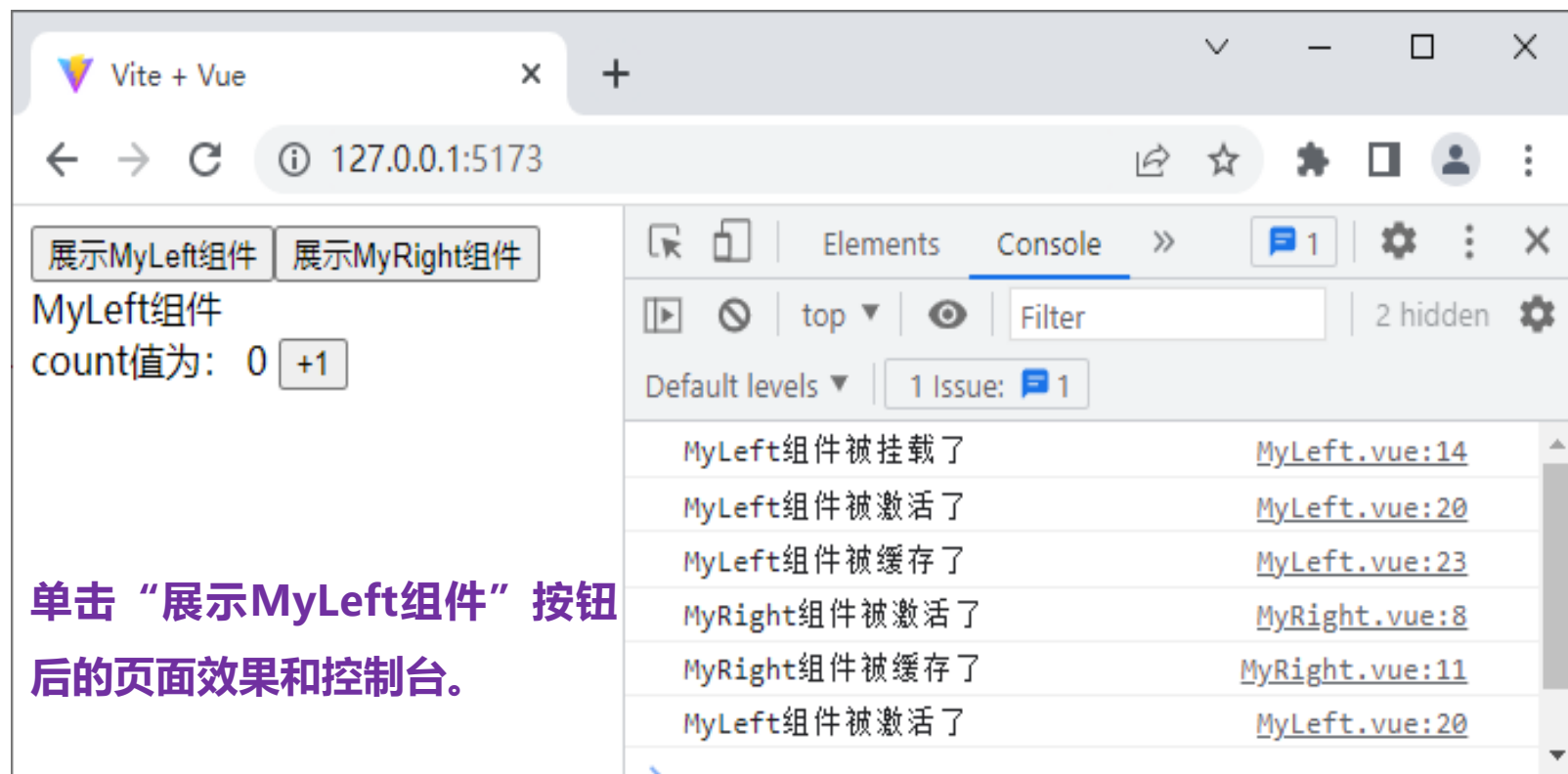
组件缓存相关的生命周期函数的使用示例

在浏览器中访问<http://127.0.0.1:5173/>。



>>> 8.5 动态组件

组件缓存相关的生命周期函数的使用示例



The screenshot shows a web browser window with the address bar at 127.0.0.1:5173. The page contains two buttons: "展示MyLeft组件" and "展示MyRight组件". Below the buttons, it displays "MyLeft组件" and "count值为: 0" with a "+1" button. The browser's developer console is open, showing a list of log messages.

展示MyLeft组件 展示MyRight组件

MyLeft组件
count值为: 0 +1

单击“展示MyLeft组件”按钮后的页面效果和控制台。

Message	File
MyLeft组件被挂载了	MyLeft.vue:14
MyLeft组件被激活了	MyLeft.vue:20
MyLeft组件被缓存了	MyLeft.vue:23
MyRight组件被激活了	MyRight.vue:8
MyRight组件被缓存了	MyRight.vue:11
MyLeft组件被激活了	MyLeft.vue:20

» 8.5 动态组件

8.5.4 KeepAlive组件的常用属性

- 在默认情况下，所有被<KeepAlive>标签包裹的组件都会被缓存。如果想要实现特定组件被缓存或者特定组件不被缓存的效果，可以通过KeepAlive组件的常用属性include、exclude属性来实现。
- KeepAlive组件的常用属性如下表所示。

属性	类型	说明
include	字符串或正则表达式	只有名称匹配的组件会被缓存
exclude	字符串或正则表达式	名称匹配的组件不会被缓存
max	数字	最多可以缓存组件实例个数

» 8.5 动态组件

8.5.4 KeepAlive组件的常用属性

- 在<KeepAlive>标签中使用include属性和exclude属性时，多个组件名之间使用英文逗号分隔，以include属性为例，语法格式如下。

```
<KeepAlive include="组件名1, 组件名2">  
  被缓存的组件  
</KeepAlive>
```

在非setup语法糖的<script>标签中定义name选项的示例代码。

```
<script>  
  export default {  
    name: 'MyComponent'  
  }  
</script>
```

注意

在使用KeepAlive组件对名称匹配的组件进行缓存时，它会根据组件的name选项进行匹配。

- 未使用setup语法糖，必须手动声明name选项；
- 使用setup语法糖，Vue会根据文件名自动生成name选项，无须手动声明name选项。

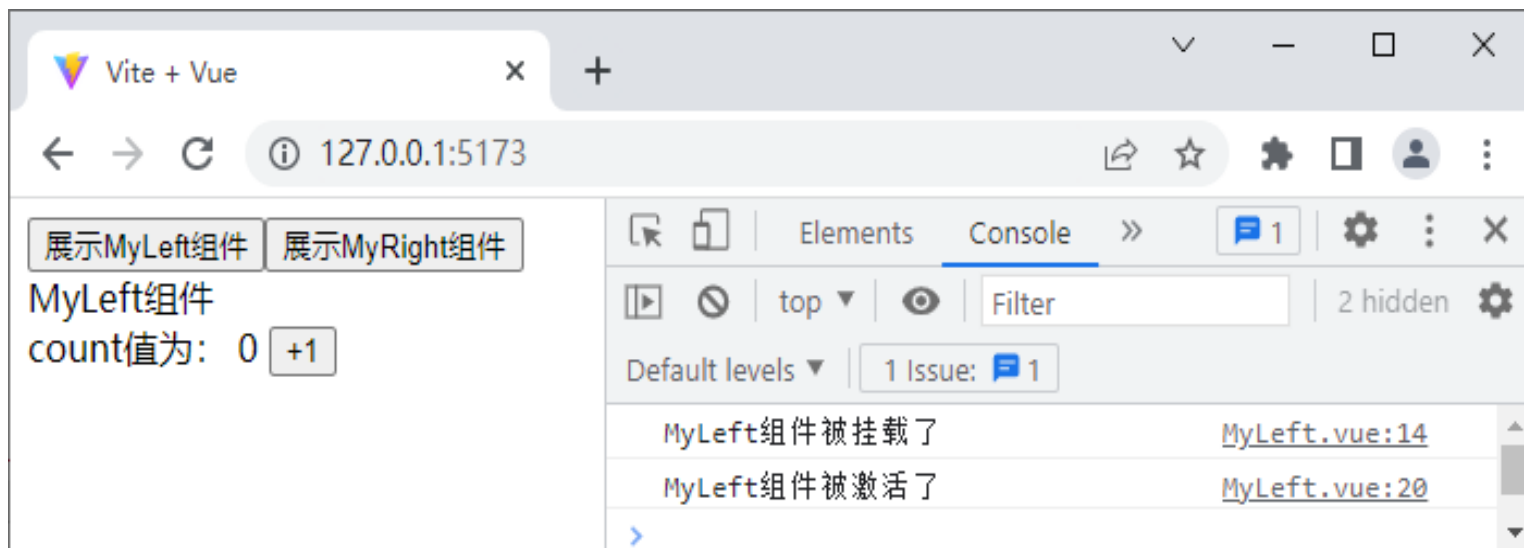
8.5 动态组件

※KeepAlive组件的include属性的使用方法示例※

①修改 src\components\DynamicComponent.vue组件中的代码，实现只缓存MyLeft组件。

```
<KeepAlive include="MyLeft">  
  <component :is="showComponent"> </component>  
</KeepAlive>
```

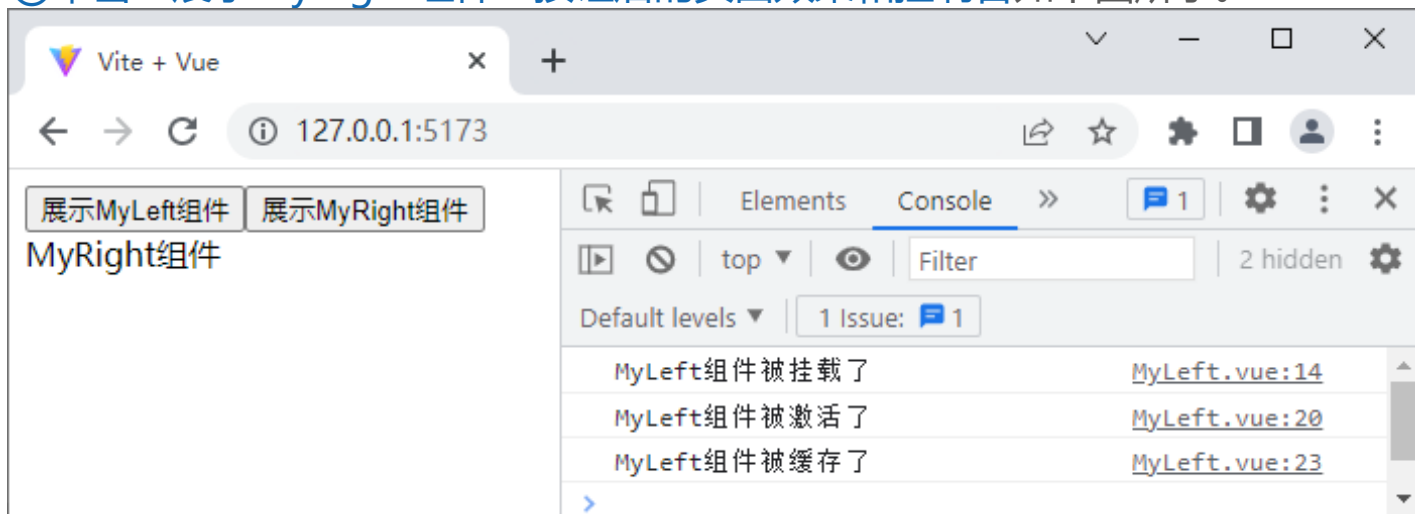
②在浏览器中访问http://127.0.0.1:5173/，初始页面效果和控制台如下图所示。



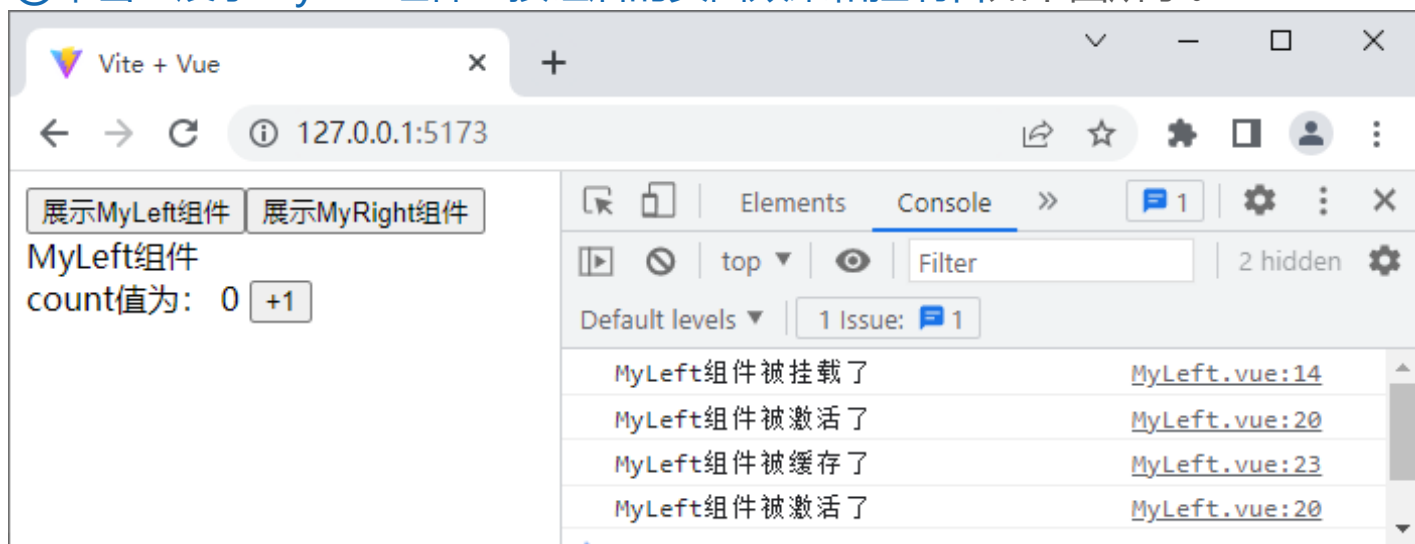
>>> 8.5 动态组件

※KeepAlive组件的include属性的使用方法示例※

③单击“展示MyRight组件”按钮后的页面效果和控制台如下图所示。



④单击“展示MyLeft组件”按钮后的页面效果和控制台如下图所示。





8.6

插槽

»» 8.6 插槽

8.6.1 什么是插槽

■ 插槽

- 开发者在封装组件时不确定的、希望由组件的使用者指定的部分。
- 插槽是组件封装期间为组件的使用者预留的占位符，允许组件的使用者在组件内展示特定的内容。
- 通过插槽，可以使组件更灵活、更具有可复用性。
- 插槽需要定义后才能使用。

8.6 插槽

8.6.1 什么是插槽

1. 定义插槽

- 在封装组件时，可以通过`<slot>`标签定义插槽，从而在组件中预留占位符。
- 假设项目中有一个MyButton组件，在MyButton组件中定义插槽的代码如下。

```
<template>
  <button>
    <slot> </slot>
  </button>
</template>
```

- 在`<slot>`标签内可以添加一些内容作为插槽的默认内容。
 - 组件的使用者没有为插槽提供任何内容，则默认内容生效；
 - 组件的使用者为插槽提供了插槽内容，则该插槽内容会取代默认内容。
- 如果一个组件没有预留任何插槽，则组件的使用者提供的任何插槽内容都会不起作用。

8.6 插槽

8.6.1 什么是插槽

2. 使用插槽

- 使用插槽即在父组件中使用子组件的插槽，在使用时需要将子组件写成双标签的形式，在双标签内提供插槽内容。例如，使用MyButton组件的插槽的代码如下。

```
<template>
  <MyButton>
    按钮
  </MyButton>
</template>
```

- 插槽内容在父组件模板中定义的，在插槽内容中可以访问到父组件的数据。插槽内容可以是任意合法的模板内容，不局限于文本。
 - 例如，可以使用多个元素或者组件作为插槽内容，示例如下。

```
<MyButton>
  <span style="color: yellow;">按钮</span>
  <MyLeft />
</MyButton>
```

»» 8.6 插槽

※插槽的使用方法示例※

①创建src\components\SlotSubComponent.vue文件——子组件。

```
<template>
  <div>测试插槽的组件</div>
  <slot></slot>
</template>
```

②创建src\components\MySlot.vue文件，用于展示插槽的相关内容。

```
1  <template>
2    父组件-----{{ message }}
3    <hr>
4    <SlotSubComponent>
5      <p>{{ message }}</p>
6    </SlotSubComponent>
7  </template>
8  <script setup>
9    import SlotSubComponent from './SlotSubComponent.vue'
10   const message = '这是组件的使用者自定义的内容'
11 </script>
```

③修改src\main.js文件。

```
import App from './components/MySlot.vue'
```

>>> 8.6 插槽

※插槽的默认内容示例※

⑤演示插槽的默认内容，实现当组件的使用者没有自定义内容时默认内容生效的效果。注释MySlot组件中插槽内容。

```
<!-- <p>{{ message }}</p> -->
```

⑥在SlotSubComponent组件中为<slot>标签提供默认内容。

```
<slot>
```

```
<p>这是默认内容</p>
```

```
</slot>
```

```
<template>
```

```
<div>测试插槽的组件</div>
```

```
<slot></slot>
```

```
</template>
```

Vite + Vue

← → ↻ ⓘ 127.0.0.1:5173

父组件-----这是组件的使用者自定义

测试插槽的组件

这是默认内容

Vite + Vue

← → ↻ ⓘ 127.0.0.1:5173

父组件-----这是组件的使用者自定义的内容

测试插槽的组件

这是组件的使用者自定义的内容

插槽内容取消注释页面效果

» 8.6 插槽

8.6.2 具名插槽

- 需要定义多个插槽：通过具名插槽来区分不同的插槽。
 - 具名插槽：给每一个插槽定义一个名称，在对应名称的插槽中提供对应的数据。
 - 具名插槽定义：利用添加name属性的<slot>标签，name属性用于给各个插槽分配唯一的名称，以确定每一处要渲染的内容。
- 定义具名插槽的语法格式如下。

```
<slot name="插槽名称"> </slot>
```

8.6 插槽

8.6.2 具名插槽

- 在父组件中，如果要把内容填充到指定名称的插槽中，可以通过一个包含v-slot指令的<template>标签来实现，语法格式如下。

```
<组件名>  
  <template v-slot:插槽名称> </template>  
</组件名>
```

- v-slot简写形式：把v-slot:替换为#。例如，v-slot:title可以简写为#title。

8.6 插槽

※具名插槽的使用示例※

①创建src\components\ArticleInfo.vue文件——文章内容模板。

```
1  <template>
2    <div class="article-container">
3      <div class="header-box"><slot name="header"></slot></div>
4      <div class="content-box"><slot name="content"></slot></div>
5      <div class="footer-box"><slot name="footer"></slot></div>
6    </div>
7  </template>
8  <style>
9    .article-container > div { border: 1px solid black; }
10 </style>
```


8.6 插槽

※具名插槽的使用示例※

②创建src\components\MyArticle.vue文件，用于提供文章数据，在MyArticle组件中导入并使用ArticleInfo组件，并在<ArticleInfo>标签中为不同插槽添加不同的信息。

```
1  <template>
2    <ArticleInfo>
3      <template v-slot:header><p>这是文章的头部区域</p></template>
4      <template v-slot:content><p>这是文章的内容区域</p></template>
5      <template #footer><p>这是文章的尾部区域</p></template>
6    </ArticleInfo>
7  </template>
8  <script setup>
9    import ArticleInfo from './ArticleInfo.vue'
10 </script>
```

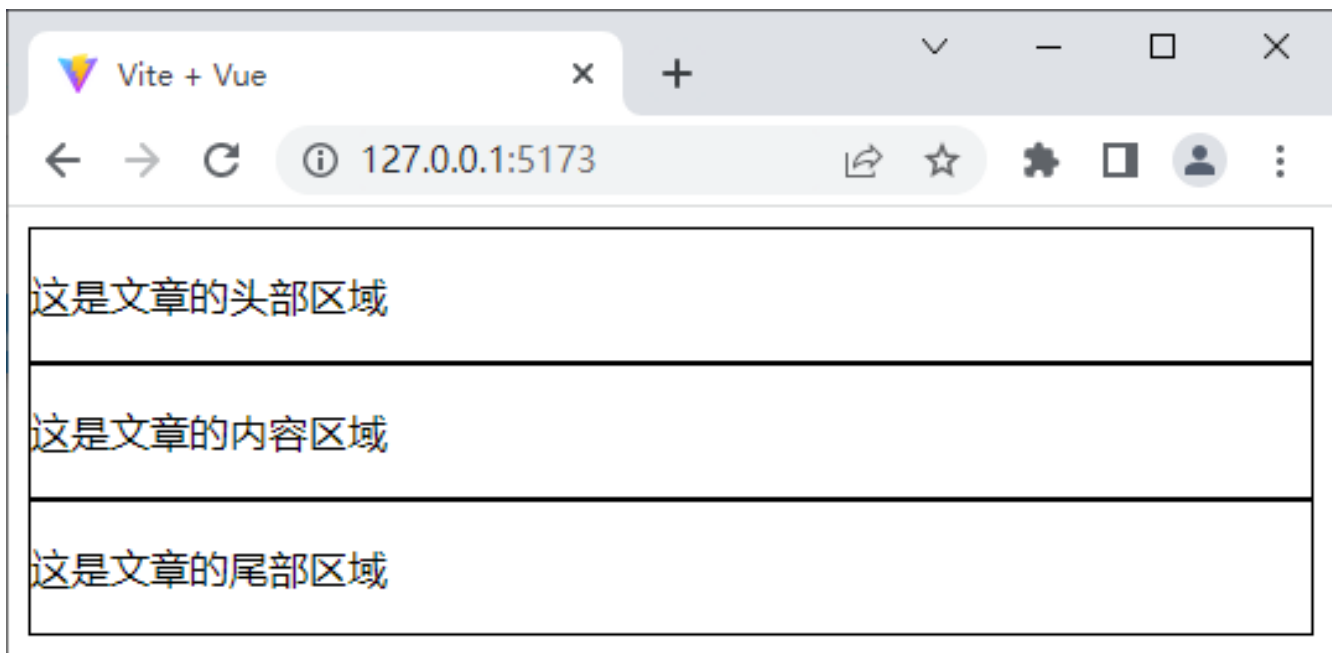
③ 修改src\main.js文件。

```
import App from './components/MyArticle.vue'
```

8.6 插槽

※具名插槽的使用示例※

在浏览器中访问<http://127.0.0.1:5173/>，使用具名插槽的页面效果。



8.6 插槽

8.6.3 作用域插槽

- 在父组件中使用子组件中定义的数据，需要通过作用域插槽来实现。
(一般情况下，在父组件中不能使用子组件中定义的数据。)
- 作用域插槽：是带有数据的插槽，子组件提供一部分数据给插槽，父组件接收子组件的数据进行页面渲染。
- 作用域插槽使用：分为定义数据和接收数据两个部分。

8.6 插槽

8.6.3 作用域插槽

1. 定义数据

- 在封装组件的过程中，可以为预留的插槽定义数据，供父组件接收并使用子组件中的数据。在作用域插槽中，可以将数据以类似传递 `props` 属性的形式添加到 `<slot>` 标签上。
- 例如，在封装 `MyHeader` 组件时，在插槽中定义数据供父组件使用，代码如下。

```
<slot message="Hello Vue.js"> </slot>
```

8.6 插槽

8.6.3 作用域插槽

2. 接收数据

按照接收数据的方式不同分为使用默认插槽和具名插槽两种。

■ 默认插槽：

如定义插槽时省略了`<slot>`标签的`name`属性，`name`属性默认为`default`。（Vue中每个插槽都有`name`属性，表示插槽的名称。）

- 父组件中可以通过`v-slot`指令接收插槽中定义的数据，即接收作用域插槽对外提供的数据。
- 通过`v-slot`指令接收到的数据可以在插槽内通过`Mustache`语法进行访问。

8.6 插槽

8.6.3 作用域插槽

■ 默认插槽:

- 例如，在父组件中使用MyHeader组件中的插槽时，通过v-slot指令的值接收传递的数据的示例代码如下。

```
<MyHeader v-slot="scope">
  <p>{{ scope.message }}</p>
</MyHeader>
```

通过v-slot接收从作用域插槽中传递的数据，scope为形参，表示从作用域插槽中接收的数据，该形参的名称可自定义。

- 作用域插槽对外提供的数据对象可以使用解构赋值以简化数据的接收过程，代码如下

```
<MyHeader v-slot="{ message }">
  <p>{{ message }}</p>
</MyHeader>
```

通过解构赋值解构对象，解构后子组件中定义的数据可以直接访问，而不是以“形参.属性”的方式访问。



解构赋值是 **ES6 (ECMAScript 2015)** 引入的一种语法，允许你从数组或对象中快速提取值，并赋值给变量。它的核心目的是简化代码，让数据提取更直观、更高效。

```
const user = {  
  name: "Alice",  
  age: 30  
};  
  
// 传统方式  
const name = user.name;  
const age = user.age;  
  
// 解构赋值  
const { name, age } = user;  
console.log(name); // "Alice"
```

8.6 插槽

8.6.3 作用域插槽

■ 具名插槽:

Vue中通过<slot>标签添加name属性来定义具名插槽，在具名插槽中也可以向父组件中传递数据。例如，在封装MyHeader组件时，向具名插槽中传入数据的语法格式：

```
<slot name="header" message="hello"> </slot>
```

- 具名插槽和作用域插槽可以作用在同一个<slot>标签上且并不冲突。<slot>标签的name属性不会作为数据传递给插槽，最终传递给组件的数据只有message属性。
- 在使用具名插槽时，插槽属性可以作为v-slot的值被访问到，基本语法格式为“v-slot:插槽名称=” 形参 “”，简写为“#插槽名称=” 形参 “”，示例如下。

```
<MyHeader>
  <template #header="{ message }">
    {{ message }}
  </template>
</MyHeader>
```


8.6 插槽

8.6.3 作用域插槽

如果在一个组件中同时定义了默认插槽和具名插槽，并且它们均需要为父组件提供数据，这时就需要为默认插槽使用显式的<template>标签来接收数据，代码如下。

```
<MyHeader>
  <template #default="{ message }">
    {{ message }}
  </template>
</MyHeader>
```

8.6 插槽

※作用域插槽的使用示例※

① 创建src\components\SubScopeSlot.vue文件，用于展示作用域插槽。

```
1  <template>
2    <slot message="Hello 默认插槽"></slot>
3    <hr>
4    <slot message="Hello Vue.js" name="header"></slot>
5    <hr>
6    <slot :user="user" name="content"></slot>
7  </template>
8  <script setup>
9    import { reactive } from 'vue'
10   const user = reactive({ name: 'xiaoyuan', age: '15' })
11 </script>
```

8.6 插槽

※作用域插槽的使用示例※

②创建src\components\ScopeSlot.vue文件，用于为作用域插槽提供数据。

```
1  <template>
2    <SubScopeSlot>
3      <template v-slot:default="scope">
4        <p>{{ scope }}</p>
5      </template>
6      <template v-slot:header="scope">
7        <p>{{ scope }}</p>
8        <p>{{ scope.message }}</p>
9      </template>
10     <template #content="{ user }">
11       <p>{{ user.name }}</p>
12       <p>{{ user.age }}</p>
13     </template>
14   </SubScopeSlot>
15 </template>
16 <script setup>
17   import SubScopeSlot from './SubScopeSlot.vue'
18 </script>
```

③修改src\main.js文件。

```
import App from './components/ScopeSlot.vue'
```

8.6 插槽

※作用域插槽的使用示例※

ScopeSlot.vue

```
1 <template>
2   <SubScopeSlot>
3     <template v-slot:default="scope">
4       <p>{{ scope }}</p>
5     </template>
6     <template v-slot:header="scope">
7       <p>{{ scope }}</p>
8       <p>{{ scope.message }}</p>
9     </template>
10    <template #content="{ user }">
11      <p>{{ user.name }}</p>
12      <p>{{ user.age }}</p>
13    </template>
14  </SubScopeSlot>
15 </template>
16 <script setup>
17 import SubScopeSlot from './SubScopeSlot.vue'
18 </script>
```

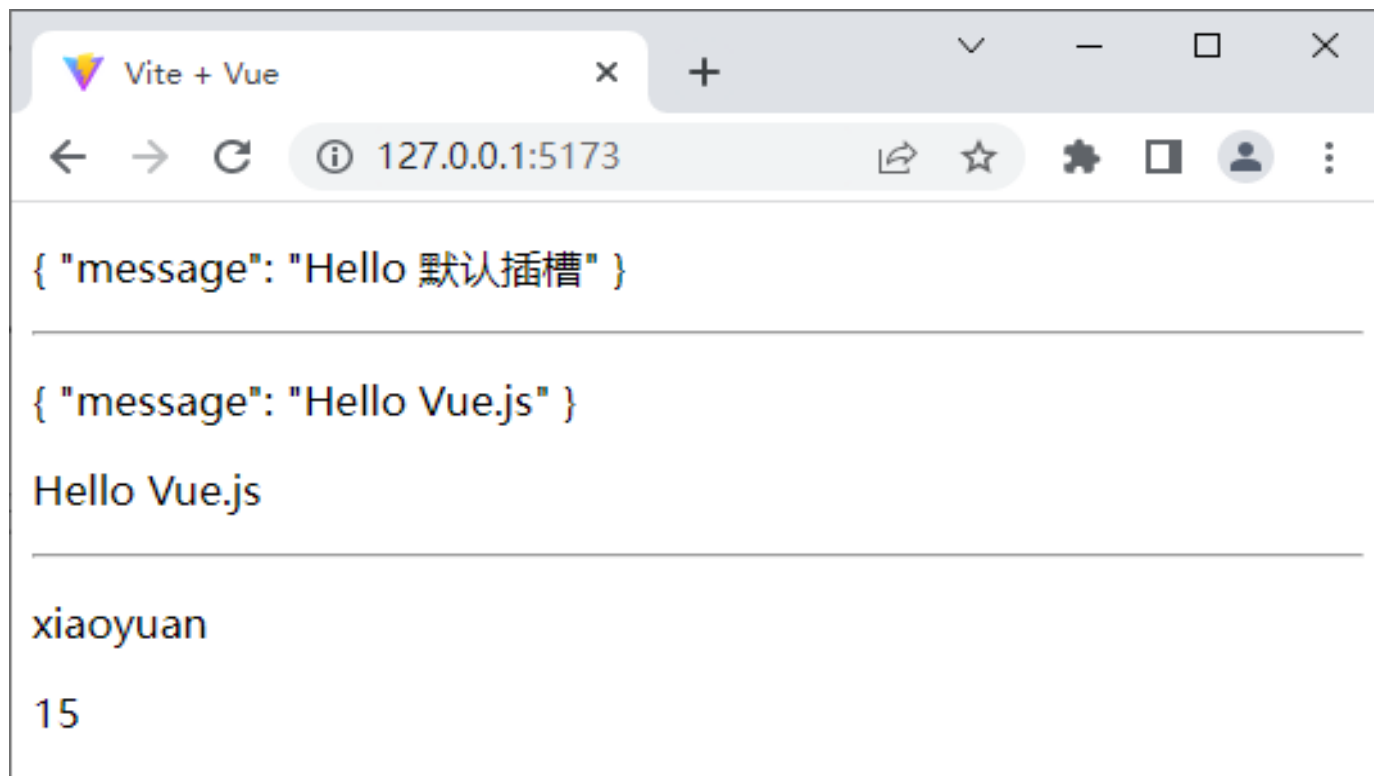
SubScopeSlot.vue

```
1 <template>
2   <slot message="Hello 默认插槽"></slot>
3   <hr>
4   <slot message="Hello Vue.js" name="header"></slot>
5   <hr>
6   <slot :user="user" name="content"></slot>
7 </template>
8 <script setup>
9 import { reactive } from 'vue'
10 const user = reactive({ name: 'xiaoyuan', age: '15' })
11 </script>
```

8.6 插槽

※作用域插槽的使用示例※

在浏览器中访问<http://127.0.0.1:5173/>，作用域插槽的页面效果。



- **作用域插槽的核心**：子组件通过 `<slot>` 绑定数据，父组件通过 `v-slot` 接收数据。
- 确保子组件正确传递了数据，且父组件解构的变量名与子组件绑定的属性名一致。



8.7

自定义指令

» 8.7 自定义指令

8.7.1 什么是自定义指令

- 当内置指令不能满足开发需求时，可以通过自定义指令来拓展额外的功能。
自定义指令的主要作用是方便开发者通过直接操作DOM元素来实现业务逻辑。
- Vue中的自定义指令分为两类。
 - 私有自定义指令
组件内部定义的指令，可以在定义该指令的组件内部使用。例如，在组件A中自定义了指令，只能在组件A中使用，组件B、C中不能使用。
 - 全局自定义指令
全局定义的指令，可以在全局使用。例如，在src\main.js文件中定义了全局自定义指令，这个指令可以用于任何一个组件。

»» 8.7 自定义指令

8.7.1 什么是自定义指令

- 一个自定义指令由一个包含自定义指令生命周期函数的参数来定义。

常用自定义指令生命周期函数表

函数名	说明
<code>created()</code>	在绑定元素的属性前调用
<code>beforeMount()</code>	在绑定元素被挂载前调用
<code>mounted()</code>	在绑定元素的父组件及自身的所有子节点都挂载完成后调用
<code>beforeUpdate()</code>	在绑定元素的父组件更新前调用
<code>updated()</code>	在绑定元素的父组件及自身的所有子节点都更新后调用
<code>beforeUnmount()</code>	在绑定元素的父组件卸载前调用
<code>unmounted()</code>	在绑定元素的父组件卸载后调用

8.7 自定义指令

8.7.1 什么是自定义指令

常用自定义指令生命周期函数的参数表

参数	说明
<code>el</code>	指令所绑定的元素，可以直接用于操作DOM元素
<code>binding</code>	一个对象，包含很多属性，用于接收属性的参数值
<code>vnode</code>	代表绑定元素底层的虚拟节点
<code>prevNode</code>	之前页面渲染中指令所绑定元素的虚拟节点

»» 8.7 自定义指令

8.7.1 什么是自定义指令

binding常用属性表

value	传递给指令的值。
arg	传递给指令的参数。
oldValue	之前的值，仅在beforeUpdate()函数和updated()函数中可用，无论值是否更改都可用。
modifiers	一个包含修饰符的对象 (如果有)。例如，在v-my-directive.foo.bar 中，修饰符对象是{ foo: true, bar: true }。
instance	使用该指令的组件实例。
dir	指令的定义对象。

» 8.7 自定义指令

8.7.2 私有自定义指令的声明与使用

未使用`setup`语法糖，可以在`directives`属性中声明私有自定义指令。

例如，声明一个私有自定义指令`color`，示例代码如下。

```
export default {  
  directives: {  
    color: {}  
  }  
}
```

说明：`color`—自定义指令的名称。`color`指令指向一个配置对象，对象中可以包含自定义指令的生命周期函数，可通过这些函数来操作DOM元素。

» 8.7 自定义指令

8.7.2 私有自定义指令的声明与使用

- 在使用自定义指令时，需要以 “v-” 开头，示例代码如下。

```
<h1 v-color>标题</h1>
```

- 如果使用setup语法糖，任何以 “v” 开头的驼峰式命名的变量都可以被用作一个自定义指令，示例代码如下。

```
<template>
  <span v-color> </span>
</template>
<script setup>
const vColor = {}
</script>
```

8.7 自定义指令

※私有自定义指令的使用方法示例※

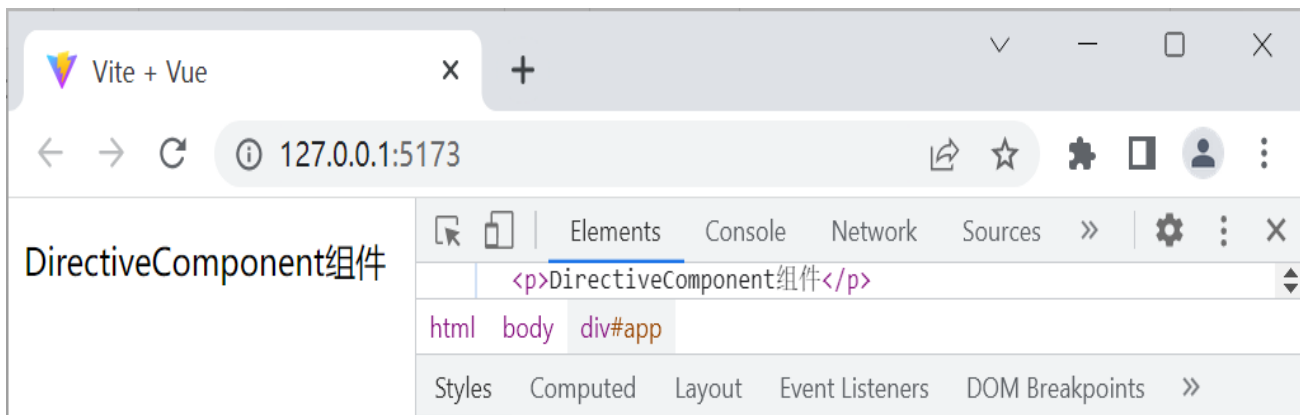
① 创建src\components\DirectiveComponent.vue文件。

```
1 <template>
2   <p v-fontSize>DirectiveComponent组件</p>
3 </template>
4 <script setup>
5   const vFontSize = {}
6 </script>
```

② 修改src\main.js文件，切换页面中显示的组件。

```
import App from './components/DirectiveComponent.vue'
```

③ 在浏览器中访问http://127.0.0.1:5173/，私有自定义指令的初始页面效果和控制台如下。



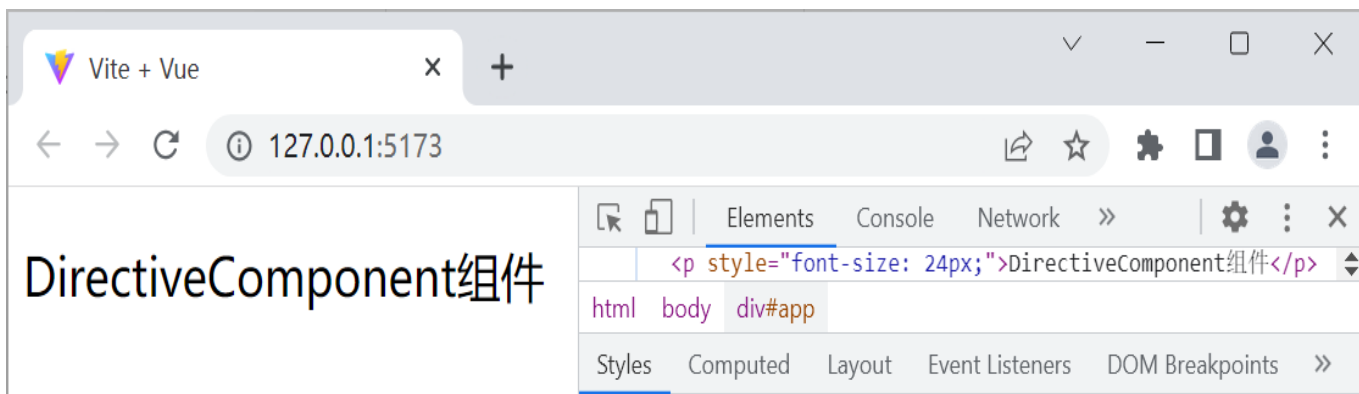
8.7 自定义指令

※私有自定义指令的使用方法示例※

④ 修改DirectiveComponent组件，添加mounted()函数，实现元素挂载完成后文本字号的改变。

```
✓ const vFontSize = {  
  ✓ mounted: el => {  
    |   el.style.fontSize = '24px'  
  }  
}
```

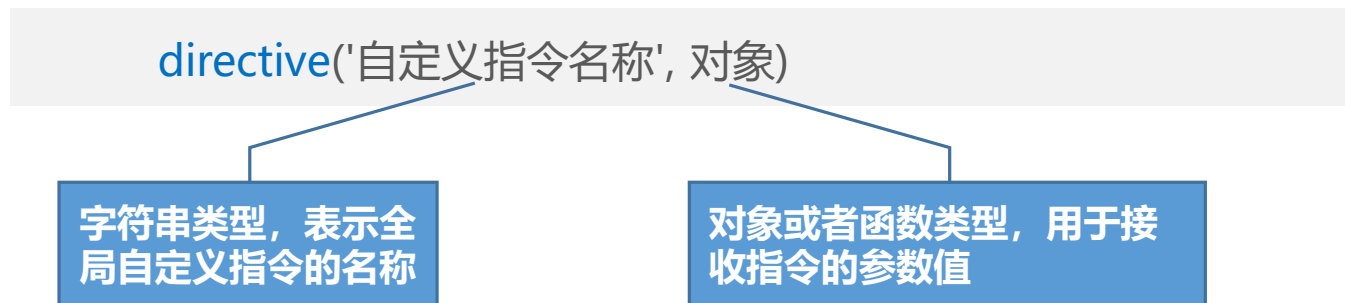
⑤ 在浏览器中访问，添加mounted()函数后的页面效果和控制台。



» 8.7 自定义指令

8.7.3 全局自定义指令的声明与使用

- 全局自定义指令需要通过Vue应用实例的`directive()`方法进行声明，语法格式如下。



- 全局自定义指令的使用方式：修改src\main.js文件，声明全局自定义指令fontSize。

```
1  import { createApp } from 'vue'
2  import './style.css'
3  import App from './components/DirectiveComponent.vue'
4  const app = createApp(App)
5  app.directive('fontSize', {
6    |   mounted: el => {
7    |     |   el.style.fontSize = '24px'
8    |   }
9  })
10 app.mount('#app')
```

» 8.7 自定义指令

8.7.4 为自定义指令绑定参数

- 使用自定义指令时，开发人员可以通过自定义指令的参数改变元素的状态，传递的参数由自定义指令的生命周期函数的第2个参数接收。
- 标签中使用自定义指令时，通过等号（=）方式为当前指令绑定参数，示例代码如下。

```
<h1 v-color="color"> </h1>
```

- 如果指令需要多个值，可以传递一个对象，示例代码如下。

```
<div v-demo="{ color: 'red', text: 'hello' }"> </div>
```


8.7 自定义指令

※自定义指令参数的使用方法示例※

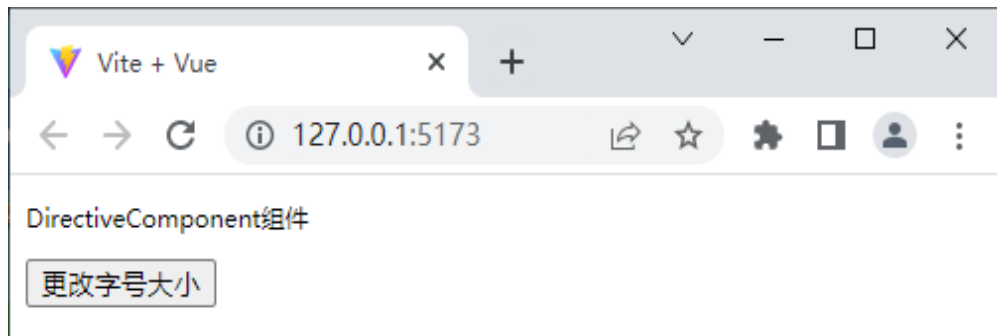
① 创建src\components\CustomDirective.vue文件。

```
1 <template>
2   <p v-fontSize="fontSize">DirectiveComponent组件</p>
3   <button @click="fontSize = '24px'">更改字号大小</button>
4 </template>
5 <script setup>
6   import { ref } from 'vue'
7   const fontSize = ref('12px')
8   const vFontSize = {
9     mounted: (el, binding) => { el.style.fontSize = binding.value },
10  }
11 </script>
```

② 修改src\main.js文件，切换页面中显示的组件。

```
import App from './components/CustomDirective.vue'
```

③ 运行程序，CustomDirective组件的初始页面效果如下图所示。



8.7 自定义指令

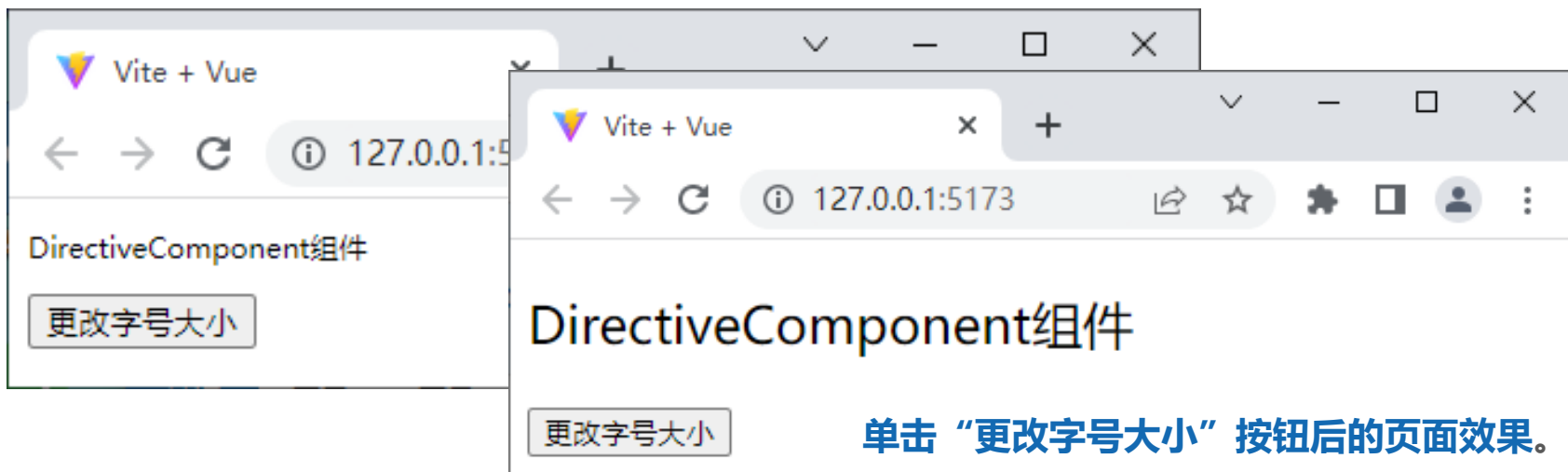
※自定义指令参数的使用方法示例※

- ④ 在自定义指令fontSize中添加updated()函数，实现自定义指令绑定的参数改变时，页面进行同步更改。

```
const vFontSize = {  
  // 原有代码.....  
  updated: (el, binding) => {  
    el.style.fontSize = binding.value  
  }  
}
```

fontSize属性值改变时，调用
updated()方法，实现字号改变

- ⑤ 在浏览器中访问http://127.0.0.1:5173/，初始页面效果如下。



单击“更改字号大小”按钮后的页面效果。

»» 8.7 自定义指令

8.7.5 自定义指令的函数形式

- 对于自定义指令来说，通常仅需要在`mounted()`函数和`updated()`函数中操作DOM元素，除此之外，不需要其他的生命周期函数。例如，上一节CustomDirective 组件中的`mounted()`函数和`updated()`函数中的代码完全相同。此时，可以将自定义指令简写为函数形式。

- 私有自定义指令简写为函数形式的示例如下。

```
const vFontSize = (el, binding) => {  
  |   el.style.fontSize = binding.value  
  |  
  }
```

- 全局自定义指令简写成函数形式的示例如下。

```
✓ app.directive('fontSize', (el, binding) => {  
  |   el.style.fontSize = binding.value  
  |  
  |})
```



8.8

引用静态资源

8.8 引用静态资源

组件中需要引用一些静态资源（图片资源、CSS代码资源等），通过项目的 `public` 目录和 `src/assets` 目录都可以存放静态资源。

1. 引用 `public` 目录中的静态资源

■ `public` 目录：存放不可编译的静态资源文件。

- 该目录下的文件会被复制到打包目录，
- 该目录下的文件需要使用绝对路径访问。

例如，在组件中引用 `public` 目录中的 `demo.png` 文件，示例如下。

```

```

>> 8.8 引用静态资源

※引用public目录中静态资源的方法示例※

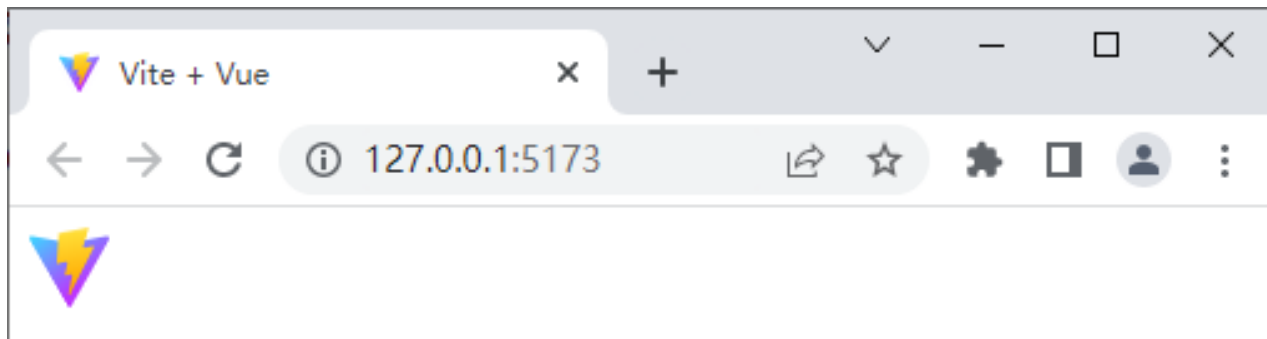
①创建src\components\Image.vue文件。

```
<template>  
    
</template>
```

②修改src\main.js文件，切换页面中显示的组件。

```
import App from './components/Image.vue'
```

运行程序，引用public目录中的静态资源的页面效果。



8.8 引用静态资源

2. 引用src\assets目录中的静态资源

- src\assets目录用于存放可编译的静态资源文件（图片、样式文件等）。
- 该目录下的文件需要使用相对路径访问。
- 引用src\assets目录中的图片：
 - ① 将图片保存到本地；
 - ② 使用import语法将图片导入需要的组件；
 - ③ 通过img元素的src属性添加图片的路径。

>>> 8.8 引用静态资源

※引用src\assets中静态资源的方法示例※

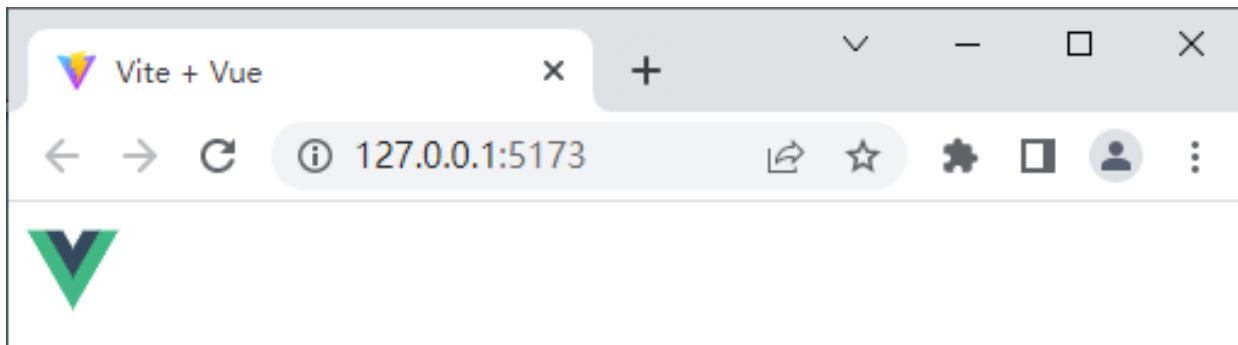
① 创建src\components\Icon.vue文件。

```
1  ∨ <template>
2    |   
3  </template>
4  ∨ <script setup>
5    import icon from '../assets/vue.svg'
6  </script>
```

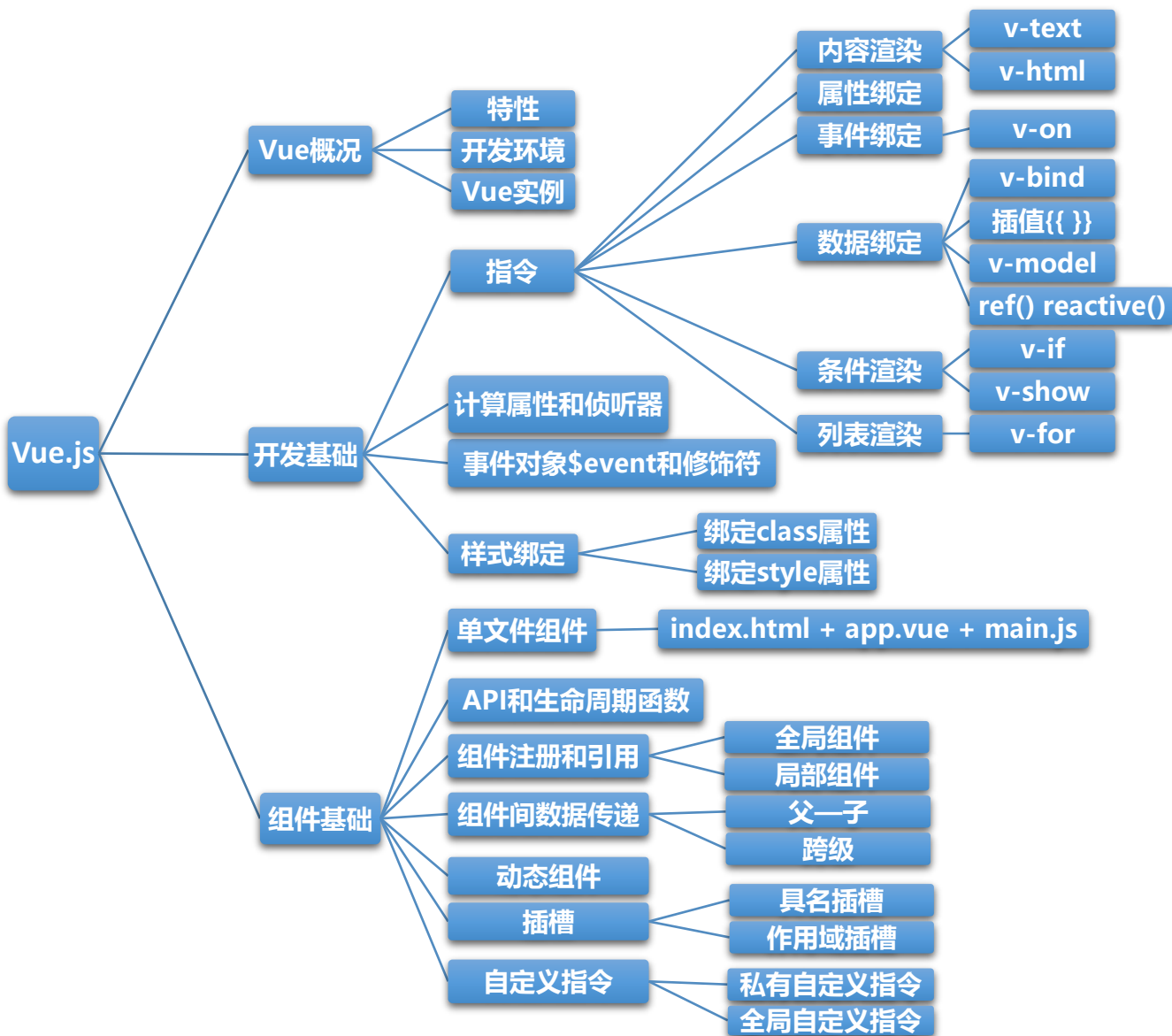
② 修改src\main.js文件，切换页面中显示的组件，示例代码如下。

```
import App from './components/Icon.vue'
```

③ 运行程序，引用src\assets目录中的静态资源的页面效果如下。



>>> 总结



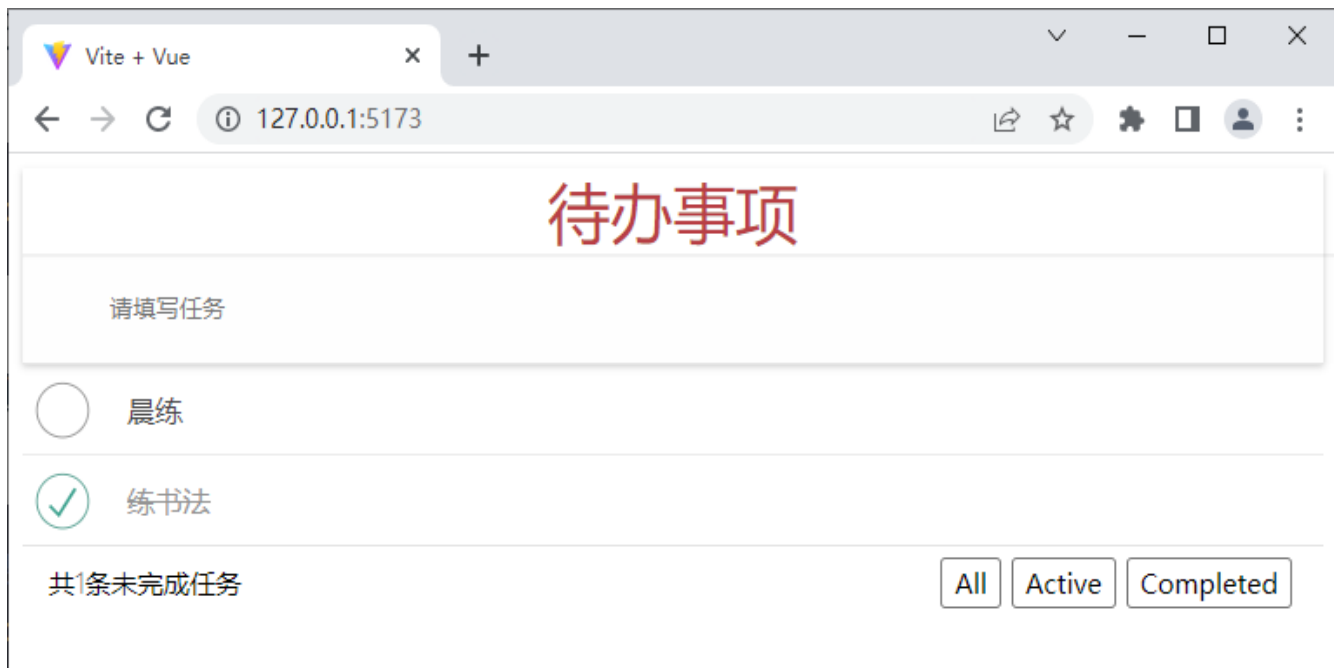
阶段案例1——待办事项

在日常生活中，人们通常倾向于对生活和工作进行提前规划，这样可以更合理地对时间进行划分，从而提高效率。本节将通过讲解待办事项案例，使读者巩固所学的Vue中组件的使用、数据共享等知识点。

阶段案例1——待办事项

1. 初始页面效果

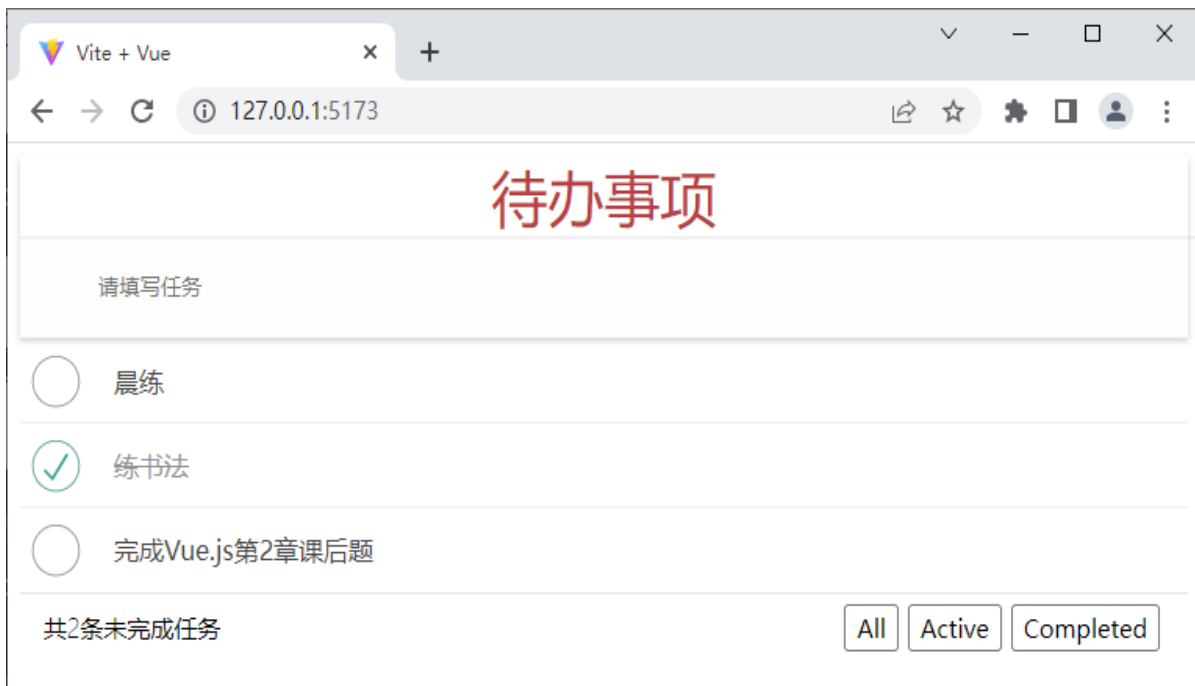
“待办事项”案例的页面结构分为上、中、下3个部分，上半部分为输入区域，中部为列表区域，下半部分为任务状态区域。



>>> 阶段案例1——待办事项

2. 新增任务

在文本框中输入内容后，在键盘上按下回车键，即可[添加任务到待办列表](#)中，添加“完成Vue.js第2章课后题”任务后的页面效果如下图所示。



>>> 阶段案例1——待办事项

3. 删除任务

当鼠标指针移入列表区域中“晨练”任务时，页面效果如下图所示。



单击“晨练”任务右侧的“x”按钮，即可删除该任务。

>> 阶段案例1——待办事项

4. 切换任务状态

在本案例中，任务状态分为全部任务、待办任务、已办任务3种。单击待办任务前的复选框，即可将待办任务状态改为已完成，并将该任务添加到已办任务中。

5. 展示任务数的条数

在任务状态区域左侧会展示任务的总条数。

>> 阶段案例1——待办事项

6. 切换任务列表

单击任务状态区域的“**All**” “**Active**” “**Completed**” 分别会切换到**全部任务**、**待办任务**、**已办任务列表**。默认情况下，任务状态区域会展示全部任务。

待办任务列表如下图所示。



>>> 阶段案例1——待办事项

7. 切换任务列表

已办任务列表如下图所示。

